


```
marp: true
theme: default
paginate: true
size: 16:9
backgroundColor: '#0B1220'
color: '#E2E8F0'
style: |
/* NotebookLM-pattern dark theme: deep navy background,
orange primary
accent (#F97316), green outcome accent (#10B981 / #34D399)
for success
metrics and outcome labels. Mirrors
talk/Portable_Agent_Context.pdf. */
section {
```

The agent is new here every time

Most coding agents enter a repo as a bright visitor with no durable map.

- They rediscover the tree.
- They infer ownership from whatever file they open first.
- They miss the invariant that decides whether an answer is safe.
- They repeat the same exploration in the next session, editor, or model.

A **cold session** — unknown files, unclear risks, expensive search — gets converted into **bounded work** — read order, named blast radius, verified output — only when the repo carries its own map.

A warm start for every coding agent

The repo carries its own map, routing rules, and verification shortcuts.

The failure is often not intelligence. It is missing repo evidence.

We already check in tests, docs, schemas, CI, runbooks, and migration history.

Agent context should be another checked-in repo artifact: the map of what is true, risky, connected, and out-of-bounds.

Portable	Reviewable	Fresh
Markdown / JSON, not vendor memory.	Lives in git. Reviewed like code changes.	Drift is a failing preflight, not a quiet footnote.

The standard is simple: read the repo map before editing the repo.

Every serious corpus needs a navigation layer

The dual-routing architecture generalizes to any file system or stable corpus an agent must explore before acting:

1. Stable datasets
2. Design systems
3. Operations runbooks
4. **Code repositories** ← we focus here today

But today, we focus on where it hurts developers the most: code repositories.

Explorable repo recall as a three-track system

agent-context works when **navigation**, **operating loop**, and **engineering** are treated as separate controls.

Track	Question	Concretely
Navigation (<i>control surface</i>)	What should load, what should not, when should it stop?	read order / risky paths
Operating Loop (<i>agent behavior</i>)	How does this agent consume repo context during real work?	trust route / verify route
Engineering (<i>systems layer</i>)	What makes prior work durable, inspectable, and reusable?	manifest / tests / CI

Outcomes when all three line up: **bounded loading** · **provenance** · **faster evidence**.

Explorable recall = an agent can re-discover the same map any time, on its own, from checked-in

Today's session: the Navigation track

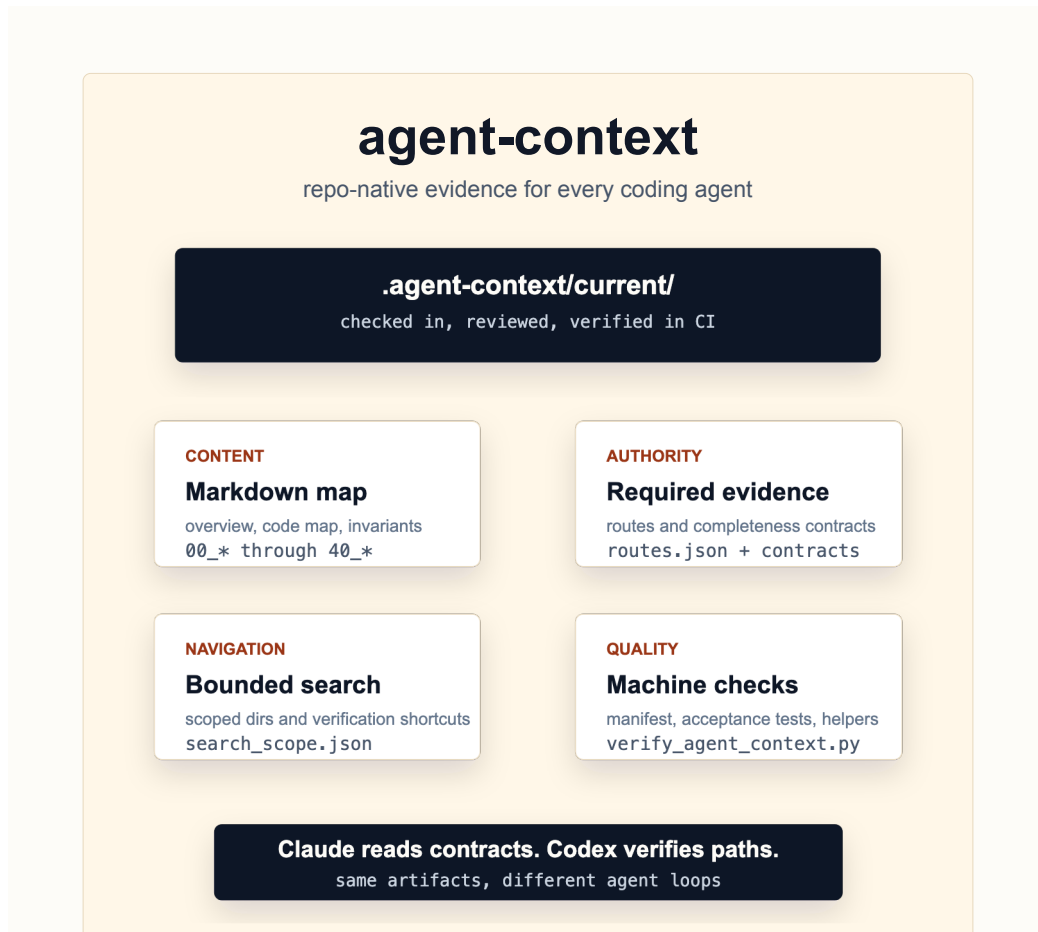
Navigation = the control surface. What the agent loads, doesn't load, and when it stops.

- **What to load** — completeness contracts that name every file family that must change together.
- **What not to load** — search scopes that bound where the agent grep-walks.
- **When to stop** — stop rules and verification shortcuts that say "you have enough to answer."

Operating Loop (how Claude vs Cursor vs Codex consume the same pack) and Engineering (manifest / tests / CI / freshness gates) are companion conversations — out of scope for this hour.

The artifact: `.agent-context/`

A committed folder next to the code. No hosted service. No hidden memory. No editor lock-in.



Same navigation design, opposite agent loops

Trust-and-Follow: reads the pack as authoritative.

Search-and-Verify: uses the pack as a starting hint, then greps to confirm.

Trust-and-Follow (Claude / Gemini)	Search-and-Verify (Cursor / Codex)
Reads completeness contracts. Stops when the contract says done.	Bounds its grep to scoped directories and cross-checks verification shortcuts.
Compressed search → fewer files opened.	Expanded proof burden → more verification, fewer dead ends.
14 → 4 files opened (<i>claude bare → structured</i>).	3 → 12 files opened (<i>cursor — but each carries a verified-against-shortcut signal</i>).

Model-agnostic by construction. The routing block (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`) points tools at `.agent-context/` before source exploration.

The engineering pipeline (a repeatable loop)

Not a library trick. An automated, checkable pipeline.

1. **Initialize** — scaffold the pack.
2. **Fill** — agent enumerates subsystems, fills templates (driven by `SKILL.md`).
3. **Verify** — structural checks, schema validation, grep-backed acceptance tests.
4. **Freshness** — fail the run when relevant code changed but context did not.
5. **Measure** — compare bare vs `structured_fresh` runs on hard tasks.

```
agent-context init . && agent-context verify . && agent-context freshness .
```

That is the meetup takeaway: **a repeatable operating loop, not a magic prompt.**

What a reviewer actually checks

The review target is the context **diff**, not the model's private memory.

- **File families** — does the pack name every file family that must change together?
- **Negative guidance** — does it say which plausible files are deprecated, generated, or near-misses?
- **Silent failures** — does it call out the thing that can break without a compile error?

Every agent starts from the same repo evidence

The routing block points tools at `.agent-context` before source exploration.

Claude Code

```
Read 00_START_HERE.md first
Load invariants and code map
Open only named source paths
```

Codex CLI

```
Read routes.json + search_scope.json
Search only relevant directories
Verify shortcut signatures
```

How to test whether it works

The methodology is deliberately boring:

Protocol piece	Why it matters
<code>bare clone</code> vs <code>structured_fresh clone</code>	Same repo, same task, only context changes.
Hard multi-hop tasks	Lookup tasks prove little; synthesis exposes wrong turns.
Grep-backed ground truth	Every expected file family is mechanically checkable.
Independent judge + audit	Self-scores are evidence, not verdicts.

Freshness is a hard experiment gate. If the pack is stale, the run fails before agents start.

Quantified evidence — Q2 2026 multi-agent rerun

252 graded answers · 48 cells · 6 repos · 4 model variants · bare vs structured_fresh.

Agent / Model	Bare yes-rate	Structured yes-rate	Δ
Claude Opus 4.7 (<i>Trust-and-Follow</i>)	80% (4.80/6)	100% (6.00/6)	+20pp
Cursor <code>claude-opus-4-7-medium</code>	89% (5.33/6)	97% (5.83/6)	+8pp
Cursor <code>composer-2-fast</code> (<i>default</i>)	61% (3.67/6)	81% (4.83/6)	+20pp
Codex CLI 0.130.0	72% (4.33/6)	78% (4.67/6)	+6pp

- Claude Opus + structured pack: 6/6 perfect across all 6 repos.
- Cursor `composer-2-fast` : biggest absolute correctness lift — +20pp.
- Production-risk → 0 for Codex (0.33 → 0.00) and Cursor Opus medium (0.50 → 0.00).

Per-agent: Claude Opus 4.7 (*Trust-and-Follow*)

Perfect 6/6 across all 6 repos under structured. The pack carries the answer; Claude trusts the contract.

Metric	Bare	Structured	Δ
Correctness yes-rate	80% (4.80/6)	100% (6.00/6)	+20pp
Risk-flag rate / 6 tasks	0.20	0.17	small
Median duration / task	65s	56s	−14%
Tool calls per correct	14.2	8.8	−38%
Files opened / task	4.5	2.7	−40%
Re-read rate (<i>same file twice = wasted attention</i>)	0.015	0.004	−73%

Per-agent: Cursor `claude-opus-4-7-medium` (*Search-and-Verify*)

219s → 78s under structured (−65%). Search-and-verify collapses when the pack guides navigation.

Metric	Bare	Structured	Δ
Correctness yes-rate	89% (5.33/6)	97% (5.83/6)	+8pp
Risk-flag rate / 6 tasks	0.50	0.00	eliminated
Median duration / task	219s	78s	−65%
Tool calls per correct	8.3	7.9	small
Files opened / task	4.8	3.9	−19%
Hops to first correct file			up — reads MORE files near the

Per-agent: Cursor **composer-2-fast** (*default model*)

Biggest absolute correctness lift — 61% → 81% (+20pp). The pack rescues a weaker model.

Metric	Bare	Structured	Δ
Correctness yes-rate	61% (3.67/6)	81% (4.83/6)	+20pp
Risk-flag rate / 6 tasks	0.67	0.50	down — still riskiest
Median duration / task	111s	112s	flat
Tool calls per correct	5.6	4.6	−18%
Files opened / task	2.9	2.6	small
Search-vs-read ratio	0.52	0.44	down
Verification-shortcut hit rate	n/a	0.17	new in structured

Per-agent: Codex CLI 0.130.0 (Search-and-Verify)

Risk flags eliminated under structured (0.33 → 0.00). 97% of the pack's files actually read.

Metric	Bare	Structured	Δ
Correctness yes-rate	72% (4.33/6)	78% (4.67/6)	+6pp
Risk-flag rate / 6 tasks	0.33	0.00	eliminated
Median duration / task	55s	126s	slower (trades wall-clock for correctness)
Tool calls per correct	11.4	9.2	−19%
Files opened / task	6.2	5.1	−18%
Pack utilization rate (fraction of			

How we measured. What it covers. What it doesn't.

- **Scope.** 252 graded tasks · 48 cells · 6 repos × 4 model variants × 2 conditions × 6 tasks. Same general-purpose template across all repos.
- **Grading.** Each cell graded by an independent Claude Code subagent — fresh context, fixed judge prompt. **No human spot-audit.** Treat numbers as directional, not certified.
- **Protocol.** Fresh-pack isolated. Every `structured_fresh` clone passed `agent-context verify` + strict `check_freshness.sh` before the agent started. Pack content authored at v0.2.0; validated at v0.3.1.
- **Telemetry caveats.** Cursor sessions in opaque SQLite — tokens/cost null in this rerun. Codex tokens are `cell_replicated` (per-cell session totals, not per-task).

Anomalies preserved (not masked): composer/structured/polyglot transient retry · opus-medium/bare/daemon hallucination retry · claude/bare/agent-chorus ran on Haiku via 5h-session fallback · `org-second-brain` skipped (pack/EXPERIMENT setup needs review).

How this relates to what you already do

You already have...	agent-context
<code>.cursorrules</code> / <code>AGENTS.md</code> / <code>CLAUDE.md</code> — single-file routing	Uses these as routing blocks pointing into the structured pack. Not a replacement.
MCP servers — runtime tool access (web, DB, internal APIs)	Different layer. agent-context is the <i>navigation map</i> ; MCP is <i>access</i> .
Vector RAG — semantic recall over chunks	Pack is <i>structural recall</i> : deterministic file lists, contracts, verification shortcuts. Pair them; don't pick.
Cursor project memory / "rules for AI"	Pack supplements, doesn't replace. Pack lives in git; IDE memory may not.

For Cursor users specifically: `.cursorrules` already exists in your repo. agent-

Tradeoffs, not magic

Tradeoff	What it means
Maintenance cost	The pack must change when relevant code changes.
Overfitting risk	Packs should route and bound search, not quote every answer.
Repo-size fit	Small repos may only need tier 1. Full packs scale to complex/monorepos.
Telemetry gaps	Cursor stores sessions in opaque SQLite; tokens/cost stay null without reverse-engineering.
Human review remains	Context improves first drafts; it does not replace code review.
Runtime cost	Structured runs are similar or cheaper <i>per correct answer</i> . The cost is human-time to author the pack.

Try this on one repo tomorrow

That is enough to decide whether the repo needs the full tier 3 pack.

1. **Pick one painful workflow** — impact analysis, auth, cache, release, migration, or docs corpus.
2. **Create tier 1** — start with code map + search scope.
3. **Add one invariant** — name the silent failure and near-miss files.
4. **Run bare vs context** — same task, same repo, two clones.
5. **Review the diff** — files opened, dead ends, missing surfaces, risk.

```
agent-context init . && agent-context verify . && agent-context freshness .
```

Make context part of the repo

Not a prompt trick. Not a vendor feature. A checked-in standard teams can review.

`init` · `verify` · `freshness` · `bare vs structured_fresh`

Today: **Navigation track**. Operating Loop and Engineering tracks are companion conversations.

Pick one repo. Run `agent-context init`. Open a PR for the diff.

github.com/cote-star/agent-context

— *Amit Prusty*